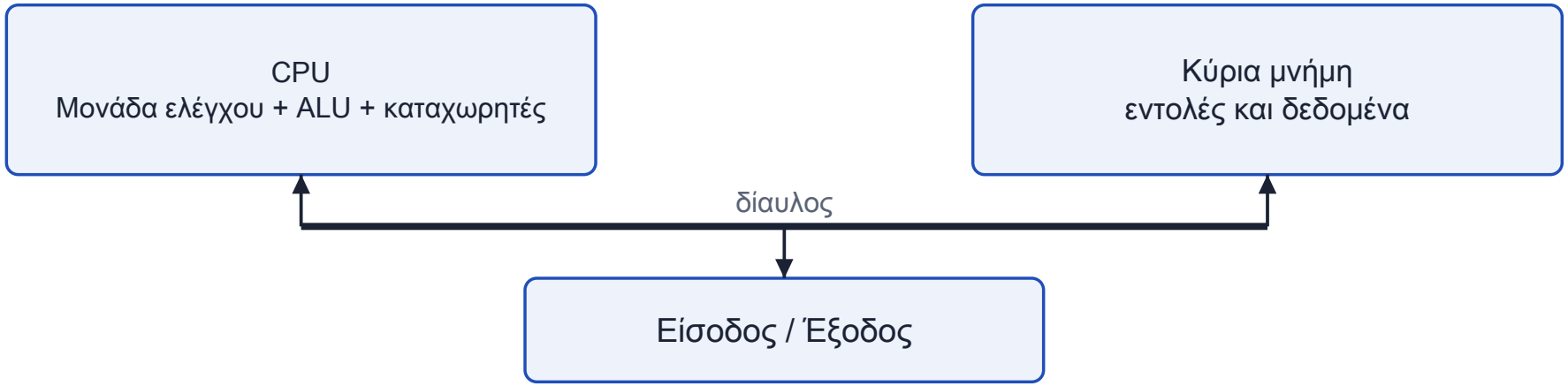
Το μοντέλο von Neumann

- Εντολές και δεδομένα βρίσκονται στην ίδια μνήμη και διαβάζονται από τον ίδιο δίαυλο.
- Η ταχύτητα μνήμης περιορίζει τον επεξεργαστή: το λέμε στενωπό von Neumann.



Ο κύκλος εντολής

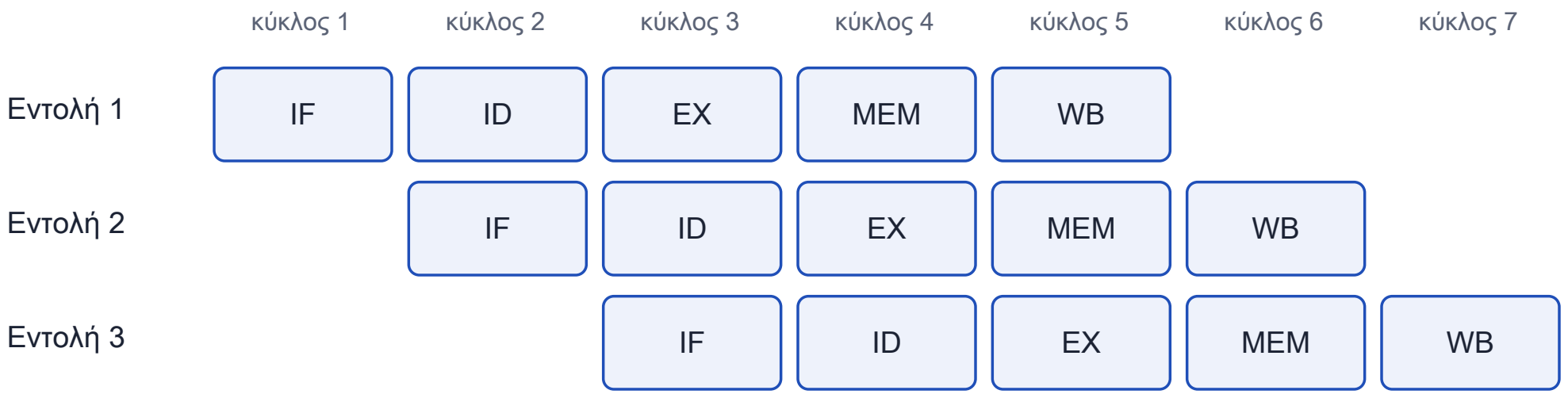
- Ανάκληση (fetch): η εντολή διαβάζεται από τη διεύθυνση που δείχνει ο PC.
- Αποκωδικοποίηση (decode): η μονάδα ελέγχου βρίσκει τι ζητά η εντολή και διαβάζει τους καταχωρητές.
- Εκτέλεση (execute): η ALU κάνει την πράξη ή υπολογίζει τη διεύθυνση.
- Πρόσβαση στη μνήμη, μόνο για load και store.
- Εγγραφή αποτελέσματος σε καταχωρητή, και ο PC προχωρά στην επόμενη εντολή.

Μέτρηση απόδοσης

- Χρόνος CPU = πλήθος εντολών × CPI × χρόνος κύκλου. Ο χρόνος κύκλου είναι 1 / συχνότητα.
- Παράδειγμα: 10^9 εντολές, CPI = 2, ρολόι 2 GHz (κύκλος 0,5 ns). Χρόνος = $10^9 \times 2 \times 0{,}5$ ns = 1 s.
- Η μεγαλύτερη συχνότητα δεν αρκεί: μετράει και το CPI, που εξαρτάται από τις εντολές.
- Νόμος του Amdahl: αν βελτιώσουμε κλάσμα f του χρόνου κατά s φορές, η συνολική επιτάχυνση είναι 1 / ((1 − f) + f / s). Για f = 0,5 και s = 2, επιτάχυνση 1,33 φορές.

Pipeline πέντε σταδίων

- Κάθε εντολή περνά από πέντε στάδια, και ξεκινά μία νέα σε κάθε κύκλο. Η καθυστέρηση μιας εντολής δεν μειώνεται· αυξάνεται η διεκπεραιωτική ικανότητα.



Κίνδυνοι στο pipeline

Κίνδυνος	Πότε εμφανίζεται	Αντιμετώπιση
Δομικός	Δύο εντολές θέλουν τον ίδιο πόρο	Αναμονή ή περισσότεροι πόροι
Δεδομένων	Μια εντολή χρειάζεται αποτέλεσμα που δεν γράφτηκε ακόμα	Προώθηση (forwarding) ή αναμονή (stall)
Ελέγχου	Δεν ξέρουμε ακόμα αν θα γίνει το άλμα	Πρόβλεψη διακλάδωσης

Μια λάθος πρόβλεψη διακλάδωσης αδειάζει τις εντολές που μπήκαν στο pipeline, με κόστος μερικών κύκλων.

Παράδειγμα κινδύνου δεδομένων

- Η sub χρειάζεται τον $t0, που η add έχει υπολογίσει στο EX αλλά γράφει στο WB. Η προώθηση το δίνει απευθείας χωρίς αναμονή.
- Μετά από lw, η τιμή είναι διαθέσιμη μόλις μετά το MEM. Ακόμα και με προώθηση χρειάζεται ένας κύκλος αναμονής (load-use hazard).

```
add $t0, $t1, $t2     # result available after EX
sub $t3, $t0, $t4     # forwarding: no stall

lw  $t5, 0($s0)       # value available after MEM
add $t6, $t5, $t1     # one stall cycle, then forwarding
```

Η ιεραρχία μνήμης

- Όσο απομακρυνόμαστε από τον επεξεργαστή, η μνήμη μεγαλώνει και αργεί. Οι τιμές είναι ενδεικτικές τάξεις μεγέθους.

Επίπεδο	Μέγεθος	Χρόνος πρόσβασης
Καταχωρητές	μερικές εκατοντάδες bytes	λιγότερο από 1 ns
Κρυφή μνήμη L1	δεκάδες KB	περίπου 1 ns
Κρυφή μνήμη L3	μερικά MB	περίπου 10 ns
Κύρια μνήμη (DRAM)	GB	περίπου 100 ns
Δίσκος SSD	TB	περίπου 100 μs

Τοπικότητα αναφορών

- Η κρυφή μνήμη δουλεύει επειδή τα προγράμματα έχουν τοπικότητα.
- Χρονική τοπικότητα: ό,τι χρησιμοποιήθηκε πρόσφατα, πιθανότατα θα ξαναχρησιμοποιηθεί.
- Χωρική τοπικότητα: μετά από μια διεύθυνση, είναι πιθανή η χρήση των γειτονικών της.
- Στη C, οι πίνακες αποθηκεύονται ανά γραμμή, άρα το a[i][j] με εξωτερικό βρόχο στο i είναι πιο γρήγορο από το αντίστροφο.

```
for (i = 0; i < N; i++)        // good: walks memory in order
    for (j = 0; j < N; j++)
        sum += a[i][j];
```

Οργάνωση κρυφής μνήμης

- Η μνήμη μεταφέρεται σε μπλοκ. Η διεύθυνση χωρίζεται σε tag | index | offset.
- Offset: διαλέγει το byte μέσα στο μπλοκ. Index: διαλέγει τη γραμμή. Tag: επιβεβαιώνει ποιο μπλοκ είναι εκεί.
- Παράδειγμα: διεύθυνση 32 bits, κρυφή μνήμη 16 KB με άμεση απεικόνιση, μπλοκ 64 bytes.
- Offset = $\log_2 64$ = 6 bits. Γραμμές = 16384 / 64 = 256, άρα index = 8 bits. Tag = 32 − 6 − 8 = 18 bits.

Συσχετιστικότητα και εγγραφές

Οργάνωση	Πού μπαίνει ένα μπλοκ	Χαρακτηριστικά
Άμεση απεικόνιση	Σε μία μόνο γραμμή	Απλή, αλλά με περισσότερες συγκρούσεις
k-δρόμων	Σε k γραμμές ενός συνόλου	Ο συνηθισμένος συμβιβασμός
Πλήρως συσχετιστική	Οπουδήποτε	Λιγότερες συγκρούσεις, ακριβός έλεγχος

Αντικατάσταση: συχνά LRU, δηλαδή βγαίνει το λιγότερο πρόσφατα χρησιμοποιημένο. Write-through: γράφουμε και στη μνήμη αμέσως. Write-back: γράφουμε στη μνήμη μόνο όταν το μπλοκ αντικατασταθεί.

Μέσος χρόνος πρόσβασης

- AMAT = χρόνος επιτυχίας + ποσοστό αποτυχίας × ποινή αποτυχίας.
- Παράδειγμα: επιτυχία σε 1 κύκλο, αποτυχία 5%, ποινή 100 κύκλοι. AMAT = 1 + 0,05 × 100 = 6 κύκλοι.
- Αν το ποσοστό αποτυχίας πέσει στο 2%, AMAT = 1 + 0,02 × 100 = 3 κύκλοι: διπλάσια ταχύτητα από μια μικρή αλλαγή.
- Τα τρία είδη αποτυχίας: υποχρεωτικές (πρώτη πρόσβαση), χωρητικότητας (δεν χωράνε όλα), σύγκρουσης (ίδια γραμμή ή σύνολο).

Ανακεφαλαίωση

- Χρόνος CPU = εντολές × CPI × χρόνος κύκλου.
- Το pipeline αυξάνει τη διεκπεραιωτική ικανότητα, όχι την ταχύτητα μιας εντολής.
- Κίνδυνοι: δομικοί, δεδομένων και ελέγχου. Το load-use θέλει έναν κύκλο αναμονής ακόμα και με προώθηση.
- Η κρυφή μνήμη αξιοποιεί τη χρονική και τη χωρική τοπικότητα.
- Διεύθυνση: tag, index, offset. AMAT = επιτυχία + αποτυχία × ποινή.